

Computação de Alto Desempenho

Henrique de Oliveira Araújo

n^o USP: 13863950

Jorge Augusto Salgado Salhani

n^o USP: 8927418

Matheus Velloso de Andrade

n^o USP: 12421725

Rafael Guimaraes Borecki de Carvalho

n^o USP: 14604271

Thiago Aragão da Silva

n^o USP: 10753062

Setembro, 2026

1 Código fonte

O código fonte utilizado como base para a construção deste relatório está disponível no GitHub e pode ser acessado pelo link abaixo:

 Computação de Alto Desempenho (Equipe) : Trabalho 1

2 Solução sequencial

Nesta seção descrevemos o formato geral do código estruturado para a solução sequencial do problema deste trabalho como modelo de propagação de incêndios.

O código pode ser dividido em 4 grandes etapas, além da leitura e validação de configurações de entrada (início de execução) e impressão de relatório final e liberação de memória (final de execução):

1. Construção de matriz linear e vetores de tempo;
2. Construção de mapa de contenção;
3. Aplicação de focos de incêndio;
4. Execução da simulação e cálculo de métricas por passo temporal;

A seguir detalhamos cada item, uma vez que serão destaques quando na paralelização da execução.

2.1 Construção de matriz linear e vetores de tempo

Cada célula da matriz é definida pela struct **Célula**.

```
typedef struct {  
    int ID_ESTADO;  
    int ID_COBERTURA;  
    int FATOR_INCENDIO;  
    int UMIDADE;  
    int TEMPO_QUEIMA;  
} Celula;
```

Com matriz linear construída no formato **Celula***, para melhor otimização quando consideramos Localidade Física.

```
Celula *matrix = malloc((size_t) configs->L * configs->C * sizeof(Celula));
```

Aqui vale ressaltar a importância da escolha de armazenamento linear ao invés de matricial (no formato **Celula****). Quando o primeiro acesso é feito para a posição 0 da matriz linear, o bloco contíguo de dados é selecionado e armazenado nas memórias cache L1/L2. Dessa forma, todos os acessos subsequentes (1 ... N) apresentará **cache hit**, com melhoria expressiva na eficiência do código.

Além disso, ao armazenar os dados linearmente, as operações podem ser feitas de forma vetorial (modelo SIMD) de modo mais eficiente. Como uma dada linha l apresenta operação idêntica e utiliza de dados presentes na sua vizinhança próxima ($l - 1$ e $l + 1$, com exceção aos casos de borda $l = 0$ e $l = L - 1$), operação vetorial pode ser empregada simultaneamente por cada núcleo de CPU.

2.2 Construção de mapa de contenção

De modo análogo podemos analisar a construção dos mapas de contenção, onde utilizamos a relação linear **int*** para armazenar o passo p em que determinada célula de índice i terá sua zona de contenção ativada. Vale notar o uso de operações inteiras, em oposição a um potencial uso de divisão e resto de divisão.

```
int *build_mapa_contencao(InputConfigs* configs, Celula* matrix) {  
    int *ativacao = (int*) malloc((size_t) configs->L * configs->C * sizeof(int));  
    if (!ativacao) return NULL;  
    for (int rowi = 0; rowi < configs->L; rowi++) {  
        for (int coli = 0; coli < configs->C; coli++) {  
            unsigned long long i = rowi * configs->C + coli;
```

```

        ativacao[i] = get_passo_ativacao_if_cell_in_zona_contencao(rowi, coli, configs);
    }
}
return ativacao;
}

```

As operações de divisão e módulo são computacionalmente caras em relação a operações de soma e multiplicação, as quais também apresentam suporte de vetorização automática (quando utilizada flag `-O3` por exemplo)

2.3 Aplicação de focos de incêndio

Utilizamos estratégia similar na aplicação dos focos iniciais de incêndio e população inicial da matriz, com geração de cobertura, fator de incêndio, estado e umidade. Embora não faça parte de execução paralela, é um importante recurso de otimização

```

bool populate_matrix(InputConfigs* configs, Celula *matrix) {
    if (!configs || !matrix) return false;
    for (int i = 0; i < configs->L; i++) {
        for (int j = 0; j < configs->C; j++) {
            unsigned long long idx = calculate_linear_matrix_index(i, j, configs->C); // row * C + col;

            int id_cobertura = generate_cobertura(configs);
            matrix[idx].ID_COBERTURA = id_cobertura;
            matrix[idx].FATOR_INCENDIO = generate_fator_incendio(id_cobertura);
            matrix[idx].UMIDADE = generate_umidade(configs);
            matrix[idx].ID_ESTADO = generate_estado(id_cobertura);
            matrix[idx].TEMPO_QUEIMA = 0;

            if (!is_valid_foco_incendio_sobre_celula_combustivel(configs, id_cobertura, i, j)) return false;
        }
    }
    return true;
}

```

2.4 Execução da simulação e cálculo de métricas por passo temporal

Para a execução da simulação propriamente dita temos 5 operações principais

1. Ativação de zonas de contenção;

2. Atualização da matriz de próximo tempo;
3. Cálculo de métricas;
4. Troca de matrizes;
5. Verificação de condição de parada.

Desta forma podemos operar com a paralelização nos instantes desejados, a citar: ativação das zonas, atualização da matriz, redução para contadores, troca de matriz de forma segura e compartilhar corretamente a condição de parada. Destacamos algumas das decisões de implementação sequencial que serão importantes quando na aplicação paralela.

A seguir, detalhamos cada uma das etapas descritas acima.

2.4.1 Ativação de zonas de contenção

Para a ativação de zonas de contenção, precisamos percorrer todo o vetor de ativação (LxC) para acioná-las em células intactas no instante p correto.

```
void activate_zonas_contencao(InputConfigs* configs, Celula* matrix, int *vetor_ativacao, int p) {
    if (!configs || !matrix || !vetor_ativacao) return;

    for (unsigned long long i = 0; i < configs->L * configs->C; i++) {
        if (vetor_ativacao[i] == p && matrix[i].ID_ESTADO == 1) matrix[i].ID_ESTADO = 4;
    }
}
```

Neste formato, temos um único dado compartilhado (passo p), acessível apenas como leitura, e a alteração em memória ocorre apenas para posições independentes (índice i do vetor **Celula* matrix**).

2.4.2 Atualização da matriz de próximo tempo

Para a atualização da matriz, operamos a transição de estados, o cálculo do potencial de ignição por célula e a contabilização do tempo de queima

```
void update_matrix(InputConfigs* configs, Celula* matrix_atual, Celula* matrix_proximo) {
    for (int i = 0; i < configs->L; i++) {
        for (int j = 0; j < configs->C; j++) {
            int idx = i * configs->C + j;
            matrix_proximo[idx].ID_ESTADO = matrix_atual[idx].ID_ESTADO;
            if (
```

```

    matrix_atual[idx].ID_ESTADO == 0 ||
    matrix_atual[idx].ID_ESTADO == 3 ||
    matrix_atual[idx].ID_ESTADO == 4
) {
    continue;
}
if (matrix_atual[idx].ID_ESTADO == 2) {
    if (matrix_proximo[idx].TEMPO_QUEIMA == 0) {
        matrix_proximo[idx].ID_ESTADO = 3;
        continue;
    }

    matrix_proximo[idx].TEMPO_QUEIMA--;
    continue;
}
int potencial_ignicao = calculate_potencial_ignicao(configs,i, j, matrix_atual);
if (potencial_ignicao < configs->LIMIAR) continue;
matrix_proximo[idx].ID_ESTADO = 2;
matrix_proximo[idx].TEMPO_QUEIMA = matrix_atual[idx].ID_COBERTURA == 2 ? 2 : 4;
}
}
}

```

A atualização da matriz apresenta análise similar à ativação de contenção. Autômatos celulares (utilizado como base na construção deste trabalho) são tipicamente utilizados para descrever fenômenos que seu estado atual apresenta dependência única e direta de seu estado anterior. Assim, ao utilizarmos duas matrizes (**matrix_atual** e **matrix_proximo**) permitimos que todo conteúdo de **matrix_atual** seja operado apenas como leitura e que os índices acessíveis por cada iteração alterem dados apenas sobre **matrix_proximo**.

2.4.3 Cálculo de métricas

Para calcularmos as métricas por instante p , precisamos contabilizar o número de células em um dado estado e o número de ignições (transição de estado $1 \rightarrow 2$, "intacta" para "em chamas"). Para facilitar o armazenamento, definimos uma struct **Metrics** no formato abaixo.

```

typedef struct {
    int PASSO;
    int COMBUSTIVEIS;
    int NAO_COMBUSTIVEIS;
    int INTACTAS;
}

```

```

int EM_CHAMAS;
int QUEIMADAS;
int CONTENCAO;
int TOTAL_IGNICOES;
double TEMPO;
} Metrics;

```

Assim, definimos um vetor **Metrics*** de tamanho P (máximo de passos da simulação) capaz de armazenar, por instante p , o acumulado de cada métrica. Como precisaremos percorrer toda a matriz e acumular o resultado (e.g. somar todas as células em estado 0), temos competição entre as variáveis acumuladas, tipicamente resolvido pelo formato **reduce**. Dessa forma, temos a seguinte função para cálculo de métricas.

```

void calculate_metrics_resultados(
    int p,
    InputConfigs* configs,
    Metrics* vetor_tempo_atual,
    Celula* matrix_atual,
    Celula* matrix_proximo
) {
    int combustiveis_iniciais = 0;
    int nao_combustiveis = 0;
    int intactas = 0;
    int total_ignicoes = 0;
    int em_chamas = 0;
    int queimadas = 0;
    int contencao = 0;
    for (unsigned long long i = 0; i < configs->L * configs->C; i++) {
        if (matrix_atual[i].ID_COBERTURA == 2 || matrix_atual[i].ID_COBERTURA == 3)
            combustiveis_iniciais++;
        switch (matrix_atual[i].ID_ESTADO) {
            case 0:
                nao_combustiveis++;
                break;
            case 1:
                intactas++;
                if (matrix_proximo[i].ID_ESTADO == 2) total_ignicoes++;
                break;
            case 2:
                em_chamas++;
                break;
            case 3:

```

```

        queimadas++;
        break;
    case 4:
        contencao++;
        break;
    default:
        break;
}
}
vetor_tempo_atual[p].PASSO = p;
vetor_tempo_atual[p].COMBUSTIVEIS = combustiveis_iniciais;
vetor_tempo_atual[p].NAO_COMBUSTIVEIS = nao_combustiveis;
vetor_tempo_atual[p].INTACTAS = intactas;
vetor_tempo_atual[p].TOTAL_IGNICOES = total_ignicoes;
vetor_tempo_atual[p].EM_CHAMAS = em_chamas;
vetor_tempo_atual[p].QUEIMADAS = queimadas;
vetor_tempo_atual[p].CONTENCAO = contencao;
}

```

Deste modo conseguimos alocar cópias privadas de cada uma das variáveis em cada thread construída para que, ao final, seja possível sincronizar em estrutura de árvore, com redução por algoritmo de complexidade $O(\log n)$.

2.4.4 Troca de matrizes

Neste momento executamos a troca de ponteiros (swap) para que a matriz de próximo estado seja compreendida como a matriz atual (que será lida e utilizada para calcular os próximos estados) e para que a matriz atual seja compreendida como a matriz de próximo estado (que terá operações de escrita sobre suas células). Caso a operação de swap seja realizada enquanto outros threads estão operando, a leitura e escrita serão feitas sobre memórias indesejadas.

Por este motivo, é necessário que todos os threads finalizem suas operações anteriores (atualização de matriz e cálculo de métricas) no passo p para que seja possível iniciar $p + 1$.

A princípio poderíamos optar pela troca de matrizes na forma a seguir

```

Celula* matrix_tmp = matrix_estado_atual;
matrix_estado_atual = matrix_proximo_estado;
matrix_proximo_estado = matrix_tmp;

```

No entanto neste formato temos dois problemas principais: o trecho de código deverá estar dentro de uma unidade de execução única (apenas um thread deve executar, após aguardo de finalização das demais) e causará

cache miss para a próxima leitura, pois a variável **matrix_proximo_estado** foi alterada e assinalada como inválida. Por estes motivos, uma possível construção desta função é dada a seguir.

```
int run_simulation(
    InputConfigs* configs,
    Celula *matrix_estado_atual,
    Celula *matrix_proximo_estado,
    int *vetor_ativacao,
    Metrics *vetor_tempo_atual,
    Metrics *vetor_proximo_tempo
) {
    Celula* matrices[2] = {matrix_estado_atual, matrix_proximo_estado};

    int p = 0;
    do {
        Celula* matrix_atual = matrices[p & 1];
        Celula* matrix_proximo = matrices[(p+1) & 1];
        vetor_tempo_atual[p].TEMPO = omp_get_wtime();
        activate_zonas_contencao(configs, matrix_atual, vetor_ativacao, p);
        update_matrix(configs, matrix_atual, matrix_proximo);
        calculate_metrics_resultados(p, configs, vetor_tempo_atual, matrix_atual, matrix_proximo);
        p++;
    } while (check_stop_condition(configs, vetor_tempo_atual[p-1]));

    return p-1;
}
```

2.4.5 Verificação de condição de parada

Como último passo, verificamos as condições de parada. Ao final de cada iteração incrementamos nosso instante ($p++$), mas, uma vez que utilizamos um loop do-while, nosso incremento de passo p deverá ser decrementado para verificação da condição de parada ($p-1$, ao fim do laço, refere-se à iteração que aconteceu no passo corrente). Aqui todos os threads deverão compartilhar do mesmo valor de p e, portanto, esta região é crítica e deve ter sua execução processada por um único thread, garantindo sincronia dos demais.

3 Análise qualitativa dos resultados

Nesta seção, analisaremos os resultados obtidos de modo qualitativo, avaliando se as alterações dos parâmetros apresentam mudanças esperadas na simulação. Para este fim, manteremos os parâmetros gerais seguintes,

conforme entrada padrão presente no arquivo disponibilizado *entrada_carga_pequena.txt*

```
400 500 80 4 2027 35
0 1 3
3 2
100 83
200 250
300 416
15 70 150 130 165
35 245 320 355 335
```

Neste arquivo de entrada podemos organizar as informações conforme as tabelas abaixo

Matriz (LxC)	400x500
Passos (P)	80
Seed	2027
Limiar	35

Focos (F) (L, C)
(100, 83)
(200, 250)
(300, 416)

Zonas (Z)
P: (LI, LF),(CI, CF)
15: [(70,150), (130, 165)]
35: [(245,320), (355,355)]

Table 1: configurações principais.

Table 2: Focos de incêndio.

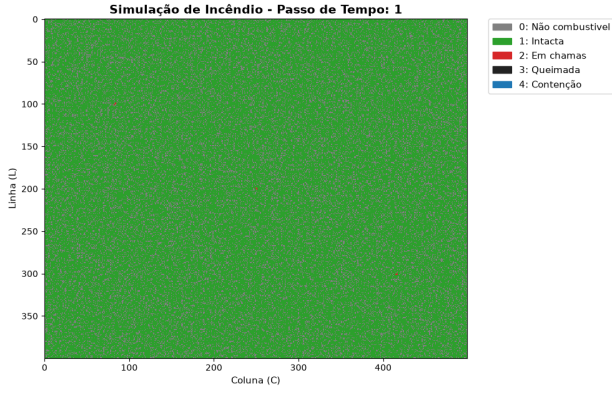
Table 3: Zonas de contenção.

A seguir, detalhamos algumas etapas da simulação para embasar as análises qualitativas realizadas.

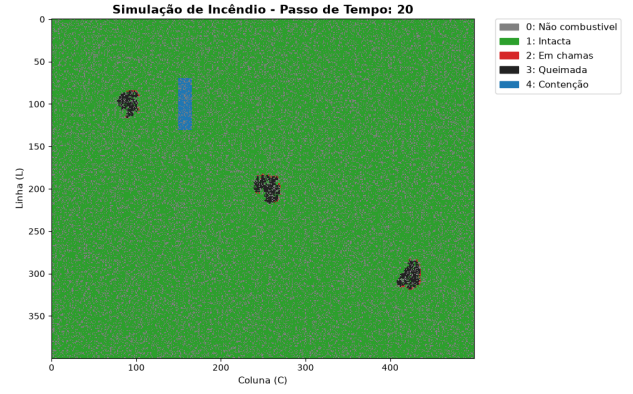
3.1 Evolução de estados

Conforme analisado na figura 1, em diferentes momentos temporais em passos p discretos representamos o mapa de estados para cada célula da matriz 400×500 .

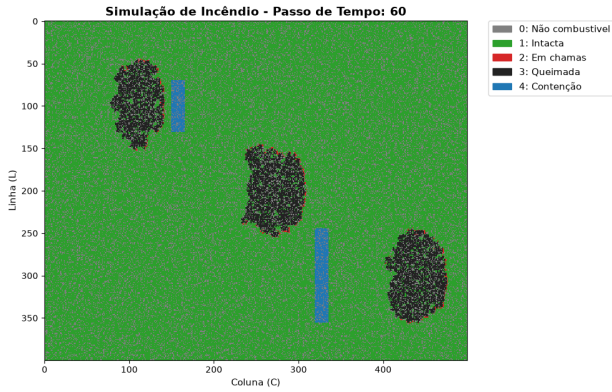
No instante de tempo $p = 1$, verificamos apenas três focos de incêndio (pelas configurações iniciais nas coordenadas (100, 83), (200, 250) e (300, 416)). Conforme avançamos para $p = 20$, notamos a criação de uma zona de contenção (azul) para a área delimitada pela região [(70, 150), (130, 165)], assim como a evolução de células adjacentes aos focos de incêndio para estados de "queimada" e "em chamas". Para $p = 60$, uma nova zona de contenção é estabelecida na região [(245, 320), (355, 355)]. Notamos também uma tendência de dispersão de células queimadas para a direita, conforme direção do vento para leste (conforme configuração de vento (0, 1), intensidade 3, que analisaremos na seção seguinte). Por fim, para $p = 80$, já no final da simulação, destacamos a manutenção da zona de contenção, com células queimadas apenas na região não englobada pela zona com início no passo $p = 15$.



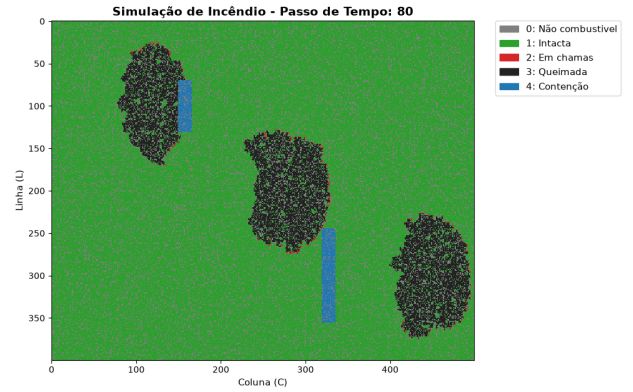
(a) Matriz de estados para $p = 1$.



(b) Matriz de estados para $p = 20$.



(c) Matriz de estados para $p = 60$.



(d) Matriz de estados para $p = 80$.

Figure 1: Mapa de estados para tempos crescentes de simulação considerando parâmetros padrão, com configurações de vento (para leste) dadas por: direção (0,1), intensidade 3. Instantes de tempo no passo (a) $p = 0$ (b) $p = 20$ (c) $p = 60$ (d) $p = 80$.

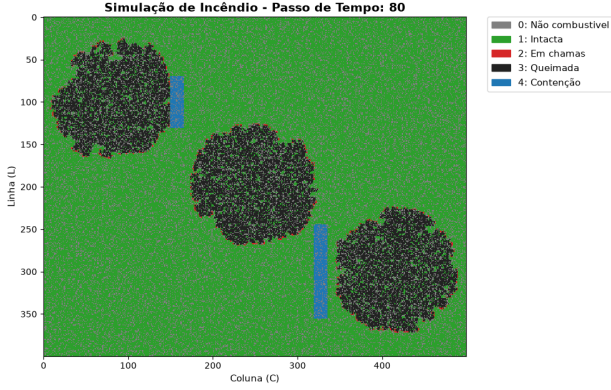
3.2 Parâmetros de vento

Neste momento, avaliaremos os efeitos da mudança de parâmetros de vento na simulação.

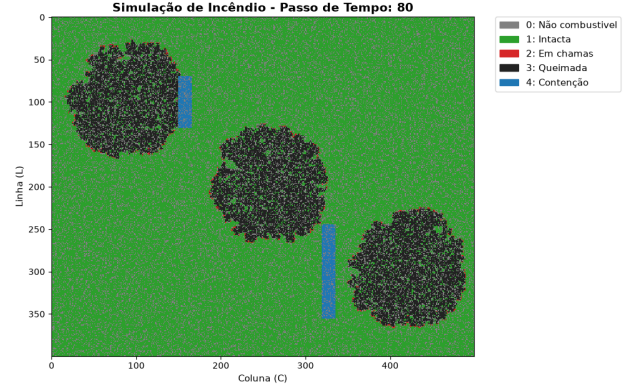
Na figura 1 mantivemos a situação final da simulação ($p = 80$) com a direção original do vento (para leste (0, 1)), mas considerando variações na intensidade do vento. Para intensidade $V = 0$, notamos a ausência de viés de propagação das células em chamas e queimadas. O fogo se alastra de modo aproximadamente circular em torno do foco inicial de incêndio. Para $V = 1$ e $V = 4$ notamos uma tendência crescente de dispersão das células em chamas e queimadas à direita dos focos de incêndio iniciais, atingindo o limite máximo possível de intensidade, com $V = 5$, onde o viés à direita é mais proeminente. As células em chamas e queimadas descreve aproximadamente uma seção de circunferência, com centro nos focos iniciais de incêndio.

Por outro lado, para a figura 3 mantivemos a intensidade padrão de ventos $V = 3$ e alteramos a direção de vento para (1,-1), sentido sudoeste, para distintos instantes de tempo com $p = 0$, $p = 20$, $p = 60$ e $p = 80$. É visível a

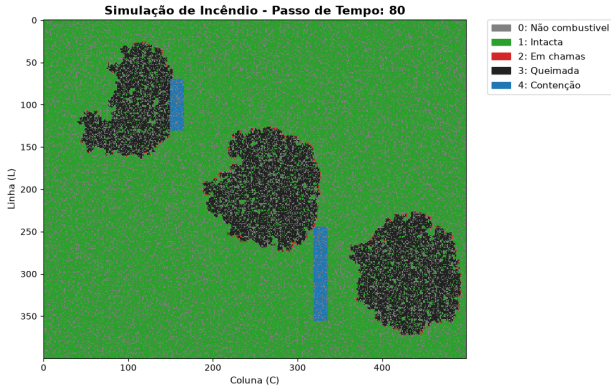
mudança de viés de propagação das células queimadas e em chamas ao longo do tempo, particularmente quando comparadas à figura 1, cujos parâmetros de vento apresentam mesma intensidade mas para leste.



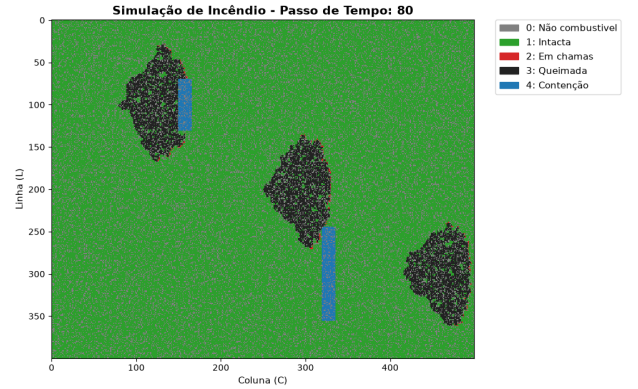
(a) Matriz de estados para intensidade de vento $V = 0$.



(b) Matriz de estados para intensidade de vento $V = 1$.



(c) Matriz de estados para intensidade de vento $V = 4$.



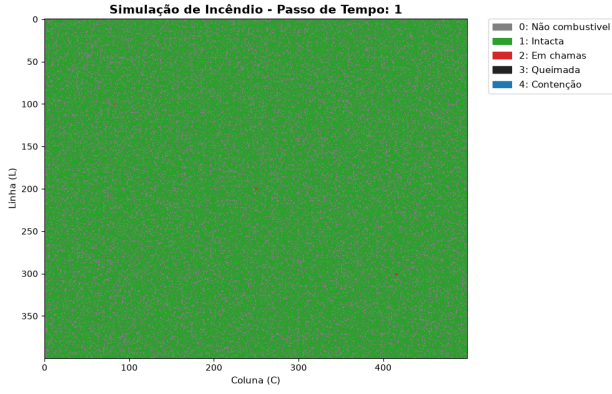
(d) Matriz de estados para intensidade de vento $V = 5$.

Figure 2: Mapa de estados para tempos fixos ($p = 80$) de simulação considerando parâmetros padrão, configurações de vento (para leste) com direção (0,1) fixa e intensidades crescentes (a) $V = 0$ (b) $V = 1$ (c) $V = 4$ (d) $V = 5$.

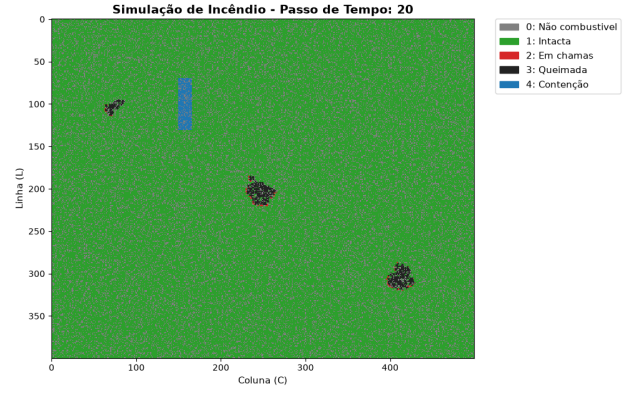
3.3 Limiar de incêndio

Analizamos agora as mudanças qualitativas na alteração do limiar de incêndio, ou seja, o valor mínimo para que uma determinada célula entre em ignição e seja alterada do estado "intacta" para "em chamas".

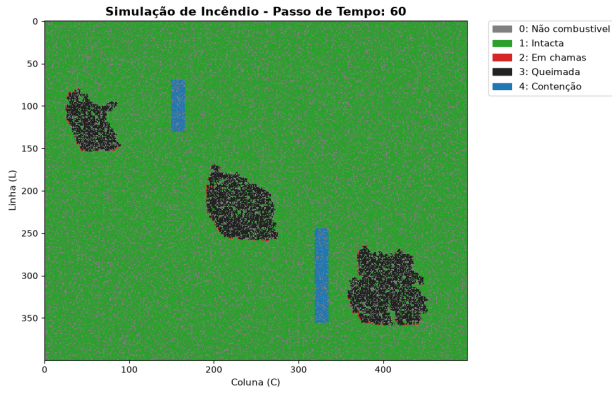
Na figura 4 mantivemos as configurações padrão apresentadas no início do capítulo, alterando o limiar de incêndio com análise do momento final da simulação $p = 80$. Para $L = 3$, notamos uma dispersão radial em relação aos focos iniciais de incêndio, uma vez que quase todas as células intactas adjacentes a uma célula em chamas serão transferidas para o estado em chamas e, em seguida, para queimadas. Conforme aumentamos o limiar para $L = 5$ e $L = 35$, menor a chance de células intactas sofrerem com as queimadas adjacentes até que, para valores bastante altos $L = 55$, algumas poucas células sofrem com as queimadas preexistentes.



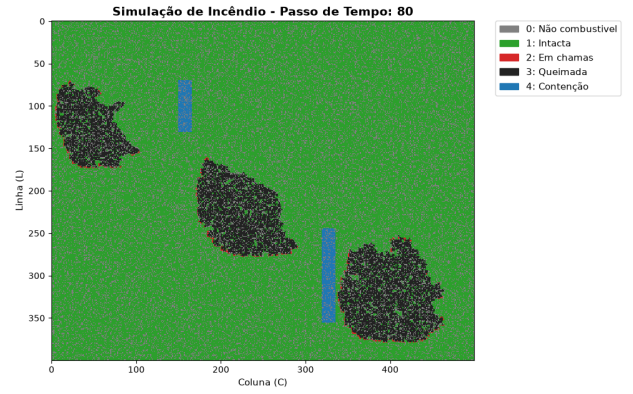
(a) Matriz de estados para o instante $p = 0$.



(b) Matriz de estados para o instante $p = 20$.



(c) Matriz de estados para o instante $p = 60$.



(d) Matriz de estados para o instante $p = 80$.

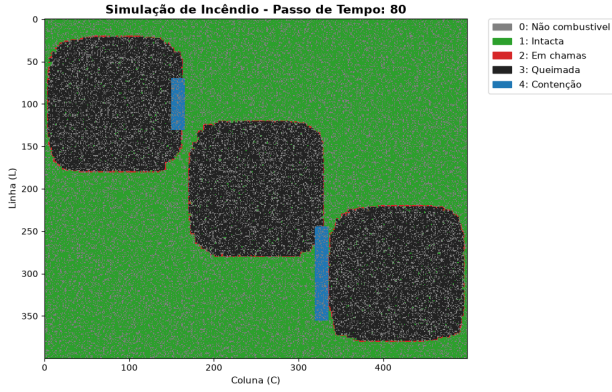
Figure 3: Mapa de estados para tempos crescentes de simulação considerando parâmetros padrão, com configurações de vento (para sudoeste) dadas por: direção (1,-1), intensidade 3. Instantes de tempo no passo (a) $p = 0$ (b) $p = 20$ (c) $p = 60$ (d) $p = 80$.

3.4 Evolução temporal

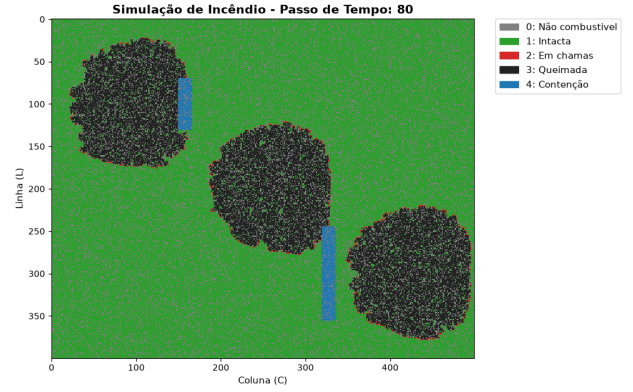
Em última análise, avaliamos a evolução temporal do número de células por estado, mantendo as configurações padrão dadas no início desta seção, com limiar de incêndio iguais a $L = 10$ e $L = 45$.

Na figura 5 Em ambos os cenários conseguimos notar o crescimento contínuo de células queimadas em oposição ao número de células intactas. Também fica evidente nos instantes de tempo $p = 15$ e $p = 35$ a aparição de múltiplas células em em estado de contenção.

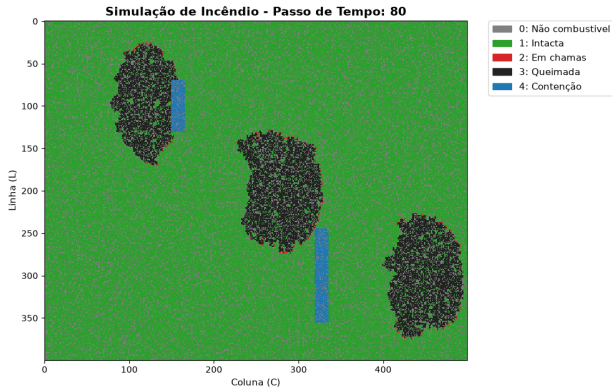
Vale também destacar, quando mantemos o limiar alto de ignição com $L = 45$, que o número de queimadas cresce em menor escala em quando comparado ao limiar $L = 10$, atingindo um máximo para $p = 64$, quando o número de células em chamas passa a reduzir ao longo do tempo.



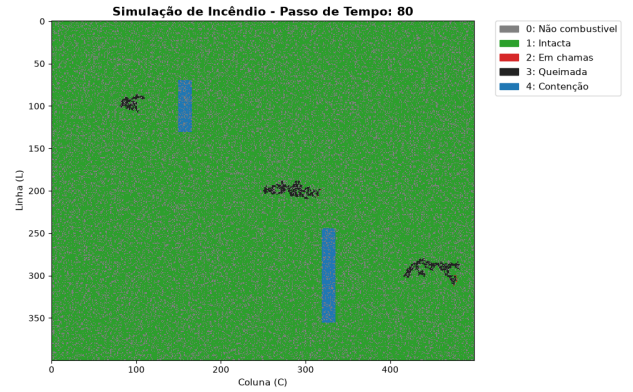
(a) Matriz de estados com limiar de incêndio $L = 5$.



(b) Matriz de estados com limiar de incêndio $L = 25$.

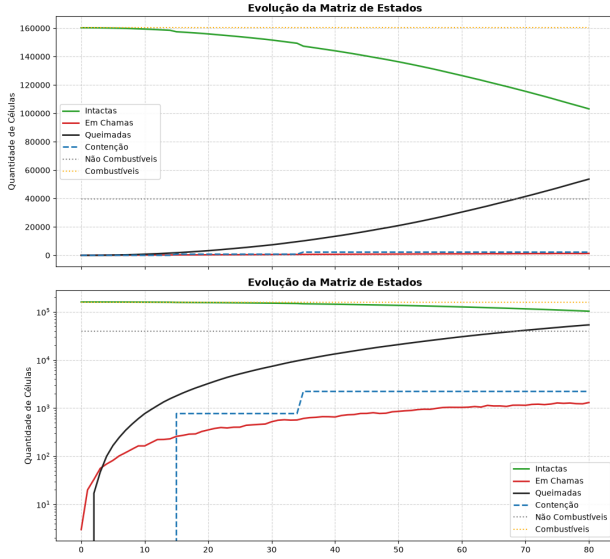


(c) Matriz de estados com limiar de incêndio $L = 35$.

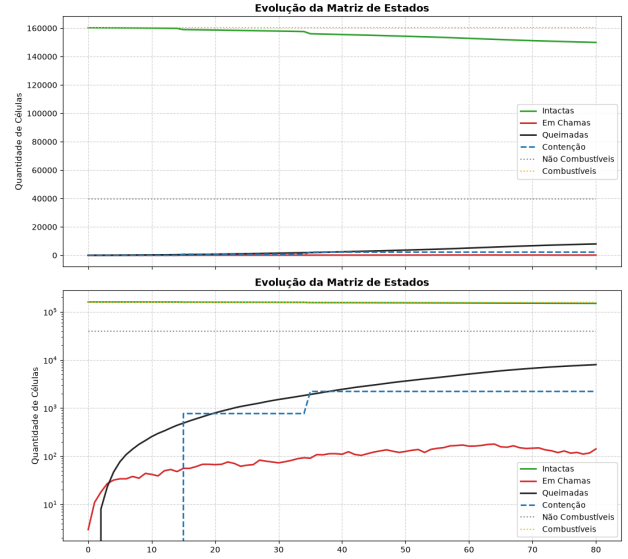


(d) Matriz de estados com limiar de incêndio $L = 55$.

Figure 4: Mapa de estados com diversos potenciais mínimos de ocorrência de um novo incêndio (limiar) considerando parâmetros padrão, com configurações de vento fixa (para leste) dadas por: direção (0,1), intensidade 3, e para tempo fixo ($p = 80$), sendo (a) limiar = 5 (b) limiar = 25 (c) limiar = 35 (d) $p = 55$.



(a) Evolução temporal do número de estados para limiar de incêndio $L = 10$.



(b) Evolução temporal do número de estados para limiar de incêndio $L = 45$.

Figure 5: Evolução temporal da matriz de estados considerando os parâmetros padrão, configurações de vento (para leste) com direção (0,1) fixa e (a) limiar = 10 e (b) limiar = 45.

3.5 Análise quantitativa dos resultados

Uma vez validada a dinâmica dos diferentes parâmetros da simulação, podemos descrever os resultados quantitativos esperados.

Para o arquivo de entrada apresentado no início da seção anterior, temos a seguinte resposta formatada

```
passos: 80
nao_combustiveis: 39704
intactas: 138809
em_chamas: 376
queimadas: 18889
contencao: 2222
total_ignicoes: 372
pico_ignicoes: 66 398
percentual_queimado: 12.02
percentual_protegido: 1.39
checksum: 35988890671224108
tempo: 0.648286
```

Utilizaremos deste formato de métricas para embasar a comparação quantitativa entre os resultados sequenciais

e paralelizados que discutiremos nas seções seguintes.